

CSA0503-DATABASE MANAGEMENT SYSTEMS

CO2 AT2

Scenario-Based Task

1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

```
SELECT
S.Student_ID,
S.Student_Name,
C.Class_Name,
T.Teacher_Name AS Class_Teacher,
G.Grade
FROM Student S
INNER JOIN Class C
ON S.Class_ID = C.Class_ID
INNER JOIN Teacher T
ON C.Teacher_ID = T.Teacher_ID
INNER JOIN Grade G
ON S.Student_ID = G.Student_ID;
```

```
mysql> SELECT
-> S.Student_ID,
-> S.Student_Name,
-> C.Class_Name,
-> T.Teacher_Name AS Class_Teacher,
-> G.Grade
-> FROM Student S
-> INNER JOIN Class C
-> ON S.Class_ID = C.Class_ID
-> INNER JOIN Teacher T
-> ON C.Teacher_ID = T.Teacher_ID
-> INNER JOIN Grade G
-> ON S.Student_ID = G.Student_ID;
```

Student_ID	Student_Name	Class_Name	Class_Teacher	Grade
201	Arun	X-A	Ramesh	A
202	Divya	X-A	Ramesh	B
203	Kavin	X-B	Priya	A
204	Nisha	X-B	Priya	A

```
4 rows in set (0.00 sec)
```

2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

```
SELECT
    D.Doctor_ID,
    D.Doctor_Name,
    COUNT(A.Appointment_ID) AS Total_Appointments
FROM Doctor D
INNER JOIN Appointment A
ON D.Doctor_ID = A.Doctor_ID
WHERE MONTH(A.Appointment_Date) = MONTH(CURDATE())
AND YEAR(A.Appointment_Date) = YEAR(CURDATE())
GROUP BY D.Doctor_ID, D.Doctor_Name
HAVING COUNT(A.Appointment_ID) > 10;
```

```
mysql> SELECT
->     D.Doctor_ID,
->     D.Doctor_Name,
->     COUNT(A.Appointment_ID) AS Total_Appointments
-> FROM Doctor D
-> INNER JOIN Appointment A
-> ON D.Doctor_ID = A.Doctor_ID
-> WHERE MONTH(A.Appointment_Date) = MONTH(CURDATE())
-> AND YEAR(A.Appointment_Date) = YEAR(CURDATE())
-> GROUP BY D.Doctor_ID, D.Doctor_Name
-> HAVING COUNT(A.Appointment_ID) > 10;
+-----+-----+-----+
| Doctor_ID | Doctor_Name | Total_Appointments |
+-----+-----+-----+
|      101 | Dr. Kumar   |          11        |
+-----+-----+-----+
1 row in set (0.01 sec)
```

3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

```
SELECT Product_ID, Product_Name
FROM Product
WHERE Product_ID NOT IN
(
    SELECT Product_ID
    FROM Order_Details
);
```

```
mysql> SELECT Product_ID, Product_Name
-> FROM Product
-> WHERE Product_ID NOT IN
-> (
->     SELECT Product_ID
->     FROM Order_Details
-> );
```

Product_ID	Product_Name
104	Monitor
105	Printer

2 rows in set (0.01 sec)

4. For a university database, write Relational Algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

```
SELECT S.Student_ID, S.Student_Name
FROM Students S
WHERE S.Student_ID IN
(
    SELECT E.Student_ID
    FROM Enrollments E
    JOIN Courses C
    ON E.Course_ID = C.Course_ID
    WHERE C.Course_Name = 'DBMS'
)
AND S.Student_ID IN
(
    SELECT E.Student_ID
    FROM Enrollments E
    JOIN Courses C
    ON E.Course_ID = C.Course_ID
    WHERE C.Course_Name = 'OS'
);
```

```
mysql> SELECT S.Student_ID, S.Student_Name
-> FROM Students S
-> WHERE S.Student_ID IN
-> (
->     SELECT E.Student_ID
->     FROM Enrollments E
->     JOIN Courses C
->     ON E.Course_ID = C.Course_ID
->     WHERE C.Course_Name = 'DBMS'
-> )
-> AND S.Student_ID IN
-> (
->     SELECT E.Student_ID
->     FROM Enrollments E
->     JOIN Courses C
->     ON E.Course_ID = C.Course_ID
->     WHERE C.Course_Name = 'OS'
-> );
```

Student_ID	Student_Name
101	Arun
104	Nisha

2 rows in set (0.01 sec)

5. Given a payroll scenario, create a view showing employees whose salary is below the department average and write a query to update their salary by 10%.

```
CREATE VIEW Below_Department_Average AS
SELECT E.Employee_ID,
       E.Employee_Name,
       E.Department_ID,
       E.Salary
FROM Employees E
WHERE E.Salary <
(
  SELECT AVG(E2.Salary)
  FROM Employees E2
  WHERE E2.Department_ID = E.Department_ID
);
```

```
mysql> CREATE VIEW Below_Department_Average AS
-> SELECT E.Employee_ID,
->        E.Employee_Name,
->        E.Department_ID,
->        E.Salary
-> FROM Employees E
-> WHERE E.Salary <
-> (
->   SELECT AVG(E2.Salary)
->   FROM Employees E2
->   WHERE E2.Department_ID = E.Department_ID
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> SELECT * FROM Below_Department_Average;
```

Employee_ID	Employee_Name	Department_ID	Salary
101	Arun	1	25000.00
104	Nisha	2	30000.00
106	Priya	3	45000.00

3 rows in set (0.01 sec)

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

```
SELECT
    C.Customer_ID,
    C.Customer_Name,
    C.Account_Number,
    T.Transaction_ID,
    T.Transaction_Date,
    T.Transaction_Type,
    T.Amount,
    B.Branch_Name,
    B.Branch_City
FROM Customers C
INNER JOIN Transactions T
    ON C.Customer_ID = T.Customer_ID
INNER JOIN Branches B
    ON C.Branch_ID = B.Branch_ID;
```

```
mysql> SELECT
-> C.Customer_ID,
-> C.Customer_Name,
-> C.Account_Number,
-> T.Transaction_ID,
-> T.Transaction_Date,
-> T.Transaction_Type,
-> T.Amount,
-> B.Branch_Name,
-> B.Branch_City
-> FROM Customers C
-> INNER JOIN Transactions T
->   ON C.Customer_ID = T.Customer_ID
-> INNER JOIN Branches B
->   ON C.Branch_ID = B.Branch_ID;
```

Customer_ID	Customer_Name	Account_Number	Transaction_ID	Transaction_Date	Transaction_Type	Amount	Branch_Name	Branch_City
101	Arun	ACC1001	1	2026-08-01	Deposit	10000.00	Chennai Main	Chennai
101	Arun	ACC1001	2	2026-08-03	Withdrawal	2000.00	Chennai Main	Chennai
102	Divya	ACC1002	3	2026-08-02	Deposit	5000.00	Chennai Main	Chennai
103	Kavin	ACC1003	4	2026-08-04	Deposit	15000.00	Coimbatore Branch	Coimbatore

4 rows in set (0.00 sec)

7. A retail store needs a daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

```
SELECT
    Category,
    SUM(Quantity * Price) AS Total_Sales
FROM Sales
WHERE Sale_Date = '2026-08-01'
GROUP BY Category
HAVING SUM(Quantity * Price) > 10000;
```

```
mysql> SELECT
->     Category,
->     SUM(Quantity * Price) AS Total_Sales
-> FROM Sales
-> WHERE Sale_Date = '2026-08-01'
-> GROUP BY Category
-> HAVING SUM(Quantity * Price) > 10000;
```

Category	Total_Sales
Electronics	102500.00
Clothing	17600.00
Groceries	15750.00

3 rows in set (0.01 sec)

8. For a library system scenario, construct Tuple Relational Calculus (TRC) expressions to find members who borrowed books from all genres.

```
SELECT M.Member_ID, M.Member_Name
FROM Members M
JOIN Borrow BR ON M.Member_ID = BR.Member_ID
JOIN Books B ON BR.Book_ID = B.Book_ID
GROUP BY M.Member_ID, M.Member_Name
HAVING COUNT(DISTINCT B.Genre) =
(
    SELECT COUNT(DISTINCT Genre)
    FROM Books
);
```

```
mysql> SELECT M.Member_ID, M.Member_Name
-> FROM Members M
-> JOIN Borrow BR ON M.Member_ID = BR.Member_ID
-> JOIN Books B ON BR.Book_ID = B.Book_ID
-> GROUP BY M.Member_ID, M.Member_Name
-> HAVING COUNT(DISTINCT B.Genre) =
-> (
->     SELECT COUNT(DISTINCT Genre)
->     FROM Books
-> );
```

Member_ID	Member_Name
101	Arun
104	Nisha

2 rows in set (0.01 sec)

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

INSERT INTO Supplier VALUES

(1, 'ABC Suppliers'),

(2, 'XYZ Traders');

INSERT INTO Product VALUES

(101, 'Laptop', 20, 55000.00, 1),

(102, 'Mouse', 100, 500.00, 2);

SELECT * FROM Product;

```
mysql> INSERT INTO Product VALUES
      -> (101, 'Laptop', 20, 55000.00, 1),
      -> (102, 'Mouse', 100, 500.00, 2);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> SELECT * FROM Product;
```

Product_ID	Product_Name	Price
101	Laptop	55000.00
102	Mouse	500.00
103	Keyboard	1200.00
104	Monitor	9000.00
105	Printer	7500.00

```
5 rows in set (0.00 sec)
```

10.Design a relational schema for an HR system and create materialized views for monthly attendance and performance.

```
CREATE TABLE Monthly_Attendance AS
```

```
SELECT
```

```
    Employee_ID,
```

```
    COUNT(*) AS Total_Days,
```

```
    SUM(CASE WHEN Status = 'Present' THEN 1 ELSE 0 END) AS Days_Present
```

```
FROM Attendance
```

```
GROUP BY Employee_ID;
```

```
CREATE TABLE Monthly_Performance AS
```

```
SELECT
```

```
    Employee_ID,
```

```
    AVG(Rating) AS Average_Rating
```

```
FROM Performance
```

```
GROUP BY Employee_ID;
```

```
SELECT * FROM Monthly_Attendance;
```

```
SELECT * FROM Monthly_Performance;
```

```
mysql> CREATE TABLE Monthly_Attendance AS
-> SELECT
->     Employee_ID,
->     COUNT(*) AS Total_Days,
->     SUM(CASE WHEN Status = 'Present' THEN 1 ELSE 0 END) AS Days_Present
-> FROM Attendance
-> GROUP BY Employee_ID;
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Monthly_Performance AS
-> SELECT
->     Employee_ID,
->     AVG(Rating) AS Average_Rating
-> FROM Performance
-> GROUP BY Employee_ID;
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Monthly_Attendance;
+-----+-----+-----+
| Employee_ID | Total_Days | Days_Present |
+-----+-----+-----+
| 101         | 2          | 2            |
| 102         | 2          | 1            |
| 103         | 2          | 2            |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Monthly_Performance;
+-----+-----+
| Employee_ID | Average_Rating |
+-----+-----+
| 101         | 4.50000        |
| 102         | 3.80000        |
| 103         | 4.90000        |
+-----+-----+
3 rows in set (0.00 sec)
```